Efficient Thinking: Tradeoffs in Game-Playing AI

Parameters vs Search vs Self-Learning — a Three-Stage Chess Study of Where Strength Comes From

Louay Alsakka · July 8, 2026

A white paper on parameter-efficient game AI.

Code & trained model: github.com/louayalsakka/efficient-thinking

The recurring theme is a set of tradeoffs: memory vs compute (Stage 1 vs 2), speed vs score (the search cascade), and diversity vs correlation (the committee) — each one a different way of spending a fixed budget to buy strength, and each capped by the same wall: the quality of the learned evaluator.

Abstract

We ask where playing strength comes from — **network capacity, search, or self-learning** — using a deliberately tiny model on modest hardware. A **3.45M-parameter (14 MB) convolutional net** plays at **~2150 Elo** as a single forward pass; adding **Monte-Carlo Tree Search (MCTS)** lifts the *same weights* to **~2800** with **zero extra parameters**. Strength is bought with **compute, not size**.

Three contributions. **(1)** A **wide→narrow MCTS cascade** that funnels the simulation budget through progressively narrower, deeper stages **matches flat MCTS at up to ~4.8× less compute per move** — our most practical result. **(2)** A direct **MCTS-vs-fixed-depth** comparison: adaptive search both beats alpha-beta at equal compute and *keeps scaling*, while fixed depth plateaus. **(3)** A **teacher-free self-learning** study (self-play, a self-referential ladder, evolution, committees): one robust positive — **agreement predicts correctness**, a confidence signal needing no oracle — and honest negatives — none of these crosses the self-play plateau, and plurality voting does not de-bias. A controlled **parameter-doubling** test agrees: doubling the net at fixed data moves strength **~0**, while 8× more *data* moves it ~+90 — capacity is the *weakest* lever in this regime. (A small-scale statement: AlphaZero-scale self-play bootstraps far past its start; we characterize the regime we could run.)

The unifying principle is **strength = evaluator × search**: search sets how *closely* you approach the evaluator's ceiling; the evaluator sets *where* that ceiling is. And across all three stages the binding constraint consistently emerged as **the quality of the information reaching the evaluator** — its training signal — rather than the machinery around it. Search *redistributes* information (cutting variance, not bias); self-play, voting, evolution and merging fail because none injects *new* information; supervision and scale succeed because they do. The method is as transferable as the numbers: **at each stage a single lever binds — only an experiment reveals which — so effort on any non-binding lever returns almost nothing.**

Headline: *A 14 MB evaluator plus adaptive search reaches ~2800-class play, and a staged MCTS cascade recovers up to 4.8× compute at little Elo cost — thinking, not growing. The organizing law is strength = evaluator × search: search extracts value, but the ceiling is the quality of the information the evaluator was given.***

> ***Read the numbers carefully.*** *Absolute Elo is measured against a Stockfish ladder and carries* **±~100 systematic uncertainty** *near the top rung — "~2800" is an efficiency indicator, not an engine-matching claim, and it is the single easiest figure for a skeptic to attack. The* **robust** *results are the relative, same-ladder ones: MCTS beats and out-scales fixed depth, the cascade holds Elo while cutting compute up to 4.8×, and every Stage-3 aggregation/self-learning method fails to beat a single model. Treat those as the paper's claims; treat 2800 as a headline.*

---

1. Introduction

Modern game AI conflates three separable questions: 1. **Open loop** — how strong is a single forward pass (pure "intuition")? 2. **Closed loop** — how much does *search* (lookahead on a learned value function) add? 3. **Self-learning** — can the system improve *itself*, with no external teacher?

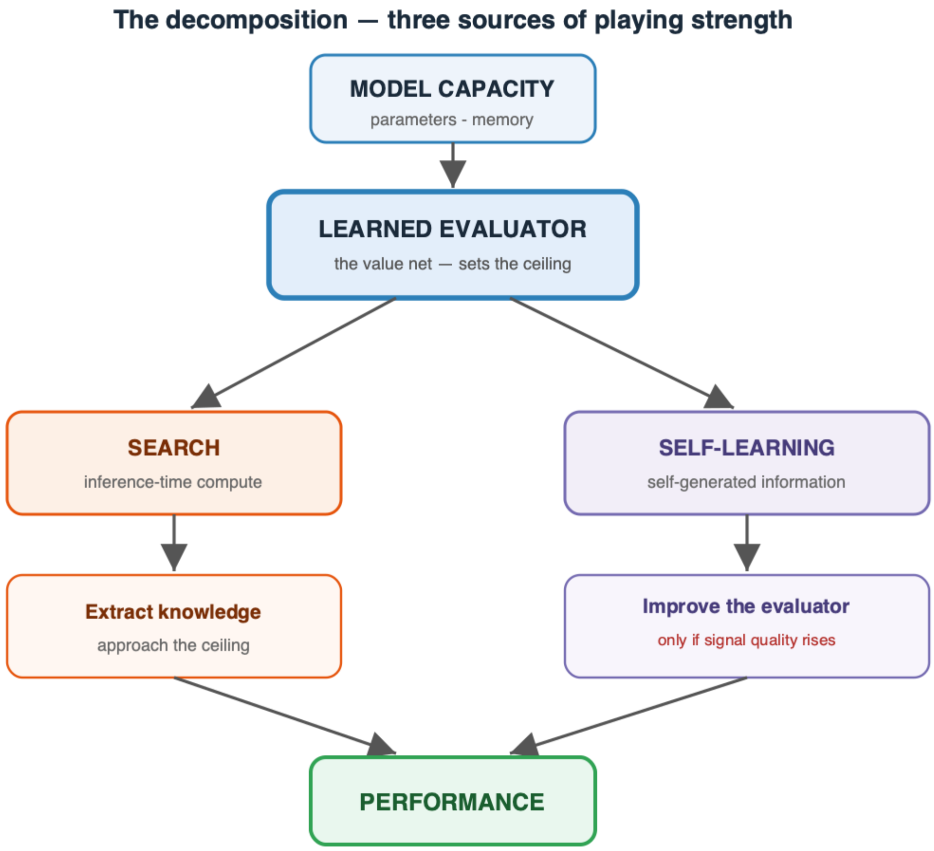
We treat these as three stages of increasing autonomy and measure each independently. The framing is explicitly **control-theoretic** (Bertsekas 2022, *Lessons from AlphaZero for Optimal, Model Predictive, and Adaptive Control*): the open-loop policy is a **feedforward controller**, closed-loop search is **model-predictive control** (receding-horizon planning with a learned terminal cost), and self-play is **iterative/adaptive learning control**. Chess is only a testbed; the goal is a *generic* recipe for sequential decision-making, and this study quantifies what each ingredient buys.

A second theme is **methodological**, and is the paper's most transferable lesson: **at any given stage, strength is gated by a *single binding bottleneck* — capacity, search,**

data, or the quality of self-generated signal — and pouring effort into any *non-binding* lever returns almost nothing. Which lever binds is not obvious a priori and shifts as you relieve each one (open-loop is capacity-bound; add search and you become search-bound until it saturates; then the evaluator — its *data*, not its parameter count in our regime — binds). The practical contribution is therefore as much a **diagnostic discipline** — run a controlled experiment to identify the binding lever before investing in it — as it is the individual numbers.

Contributions. - Our headline method — a search-efficiency result: a wide→narrow MCTS **cascade** that funnels the simulation budget through progressively narrower, deeper stages, matching flat MCTS at up to **4.8× less compute per move** with a clean score/speed trade-off curve. - A parameter-efficiency result: **~2800 Elo from 3.45M params** via search, at constant memory. - A direct **MCTS-vs-fixed-depth** result: adaptive search beats and out-scales fixed depth. - An **architecture-beats-scale** finding (convolution ≫ MLP at equal data), with topology sweep. - A reproducible **negative result** on small-scale self-play (plateaus below supervision). - A **teacher-free self-learning** study: agreement is a validated **confidence meter**, but self-play, a self-referential ladder, **evolution**, and plurality-voting committees all fail to cross the plateau — a set of clean negative results with a methodological warning (noisy fitness manufactures phantom gains).

The paper's roadmap in one picture — the three sources of strength, all feeding a single learned evaluator:



2. Problem Setup and Methods

2.1 Representation

- **Position → move** as a 4096-way (64×64 from–to) classification. Promotions default to queen.
- **Side-to-move normalization:** the board is always presented from the mover's perspective (mirror + color-swap for Black), so one network serves both players.
- **Encodings:** *onehot* = 64 squares × 12 piece planes + 5 meta bits = **773 floats** (used throughout). For convolutions the 773-vector reshapes to **8×8×12 piece planes** + 5 meta planes = 17 input channels. We compared this to a compact *packed* encoding (one integer code per square, 64+5 = **69 floats** — a 4-bit-per-square board). On an identical MLP, budget, and data, **one-hot is decisively stronger — 84.9 vs 142.5 mean centipawn-loss** — because a network learns clean categorical piece-type features far more readily than an arbitrary *ordinal* code (where code 7 vs 8 carry a spurious "closeness"), and, crucially, convolution *requires* per-piece-type planes, which a packed 8×8×1 integer board cannot provide. Packed is a compact-but-harder-to-learn ablation; one-hot is the workhorse.
- **No side-to-move feature:** the turn is not a bit — the board is *re-oriented* so the side to move is always "White" (mirror + color-swap for Black), so one evaluator serves both players and the turn is carried by the orientation. (This is why `Eval(P)` equals `Eval(mirror(P))` exactly, and why the value of the move can be read off as `Eval(B) + Eval(null-move) - 1` — the deviation from that mirror symmetry; measured mean tempo ≈ +0.15, up to +0.77 in tactical shots and negative in zugzwang.)

2.2 Labels and objective

- **Supervised data:** the full public Lichess cloud-eval database — **394,669,566 positions** (79 shards), each with Stockfish multi-PV win-probabilities.
- **Hard objective:** cross-entropy to the single best move (bounded by imitation).
- **Soft objective:** cross-entropy to an *advantage-weighted distribution* over legal moves (`softmax(vᵢ/τ)`). Soft is not bounded by copying one move and generalizes better.

2.3 Evaluation

- **Elo via a Stockfish ladder** (random baseline, then SF at set UCI_Elo rungs), 20–80 games/rung; Elo fit by maximum-likelihood against the known rung ratings.
- **Methodological caveat (important):** if the player beats the top rung, the Elo estimate **compresses/extrapolates**, so we raised the ladder as the player improved (SF-1700 → 2100 → 2700 → 3000). Absolute numbers carry systematic uncertainty; **relative gains measured on the same ladder are the robust results** (treat absolutes as ±100). Where two methods are compared, they are always run on the *same* ladder and seeds.

2.4 Hardware

- Two Apple-Silicon **Mac Studios (M3 Ultra, 256 GB each)**, MLX framework, 40 Gbps bridge.

2.5 Reproducibility (evaluation protocol)

Every Elo/CPL figure in this paper uses a fixed protocol so numbers are comparable across methods: - **Engine:** Stockfish 18 as both the ladder opponent and the CPL oracle; opponents run at fixed `UCI_Elo` rungs, movetime **0.03–0.04 s** (stated per experiment). CPL uses Stockfish at **fixed depth 12** (depth-limited, so CPU contention slows but does not weaken it). - **Ladder:** a random-mover anchor plus `UCI_Elo` rungs (e.g. 1700/2000/2300, 2400/2700/3000); Elo is the maximum-likelihood fit vs the known rung ratings. **Games per rung: 20–40** (stated per table). The reported margin is a crude $\pm 400/\sqrt{n}\cdot 2$ (≈$\pm$89 at 20 games/rung, ≈$\pm$100 typical) — treat all absolutes as **±100** and rely on *relative* same-ladder deltas. - **Openings:** sampled from the public Lichess PGN (`2013-01`), replayed to plies 6–16 for a diverse, balanced start book; both colors played from each opening. - **Seeds:** default seed 0 (stated where varied); openings, ladder, and match seeds are fixed so a method's eval is reproducible, and any two methods compared are always run on the **same** ladder, openings, and seeds. - **Known caveats:** (i) the ladder **compresses** once the player beats the top rung — we raised the ladder as strength grew, and near-top absolutes carry extra uncertainty; (ii) at movetime 0.03–0.04 s Stockfish plays **below** its nominal `UCI_Elo` , so the ladder is internally consistent but not calibrated to over-the-board Elo. Code, the trained model, and the exact scripts are in the repository.

---

3. Stage 1 — Open Loop (single-pass policy)

3.1 Topology sweep — architecture beats scale

At fixed moderate data (18M positions) we swept eight architectures:

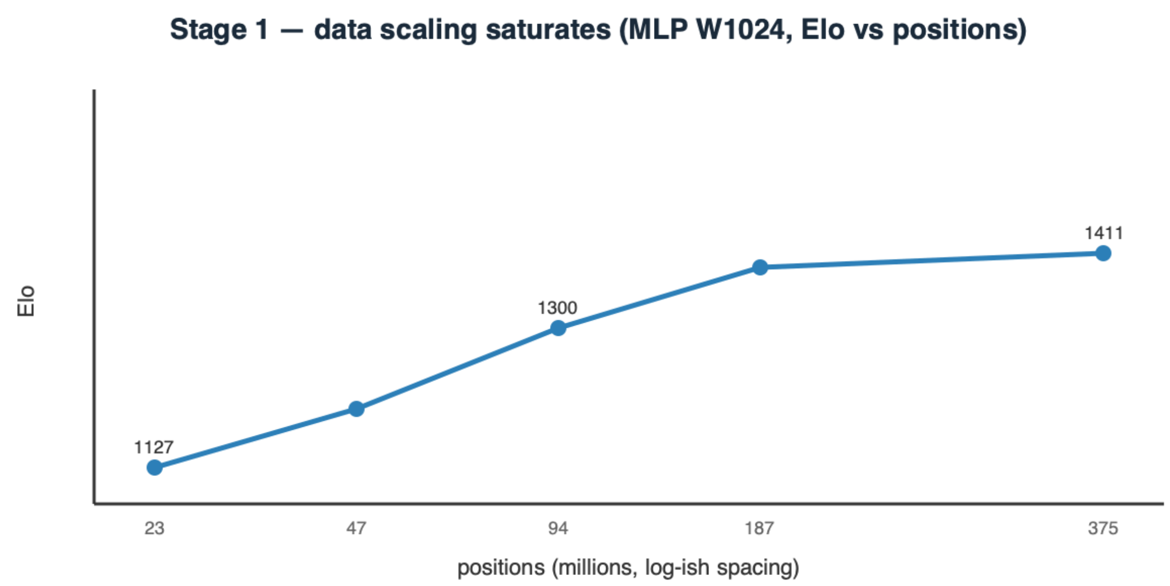
Topology	params	Elo	note
conv (64×10 / 96×8)	2.8–3.4M	**1476**	best & most efficient
constant-width MLP (1024×6)	10.2M	1108–1233	best plain MLP
dual-path (wide+deep, gated)	2.8M	1082	ties MLP at 3.5× fewer params
factored head (from/to)	6.2M	782	−450
funnel (1024→64)	1.8M	582	narrowing hurts
pyramid (64→1024)	5.0M	431	bad

Topology	params	Elo	note
bottleneck (512→8)	0.5M	340	broken

Topology conclusion. Only two shapes are competitive — **constant-width** and **convolution** — and every *taper, factoring, or exotic bottleneck loses badly*. The reason is inductive bias: a taper or bottleneck destroys positional information before the head can use it, while convolution **reuses local tactical patterns across the board via weight sharing**, so it learns a motif once instead of relearning it per square. The practical rule that falls out: **pick the prior that matches the domain's structure (spatial locality) rather than adding raw width.** A 3.45M-param conv matched a 14.4M-param MLP using **22× less data** — architecture, not parameter count, set the strength.

3.2 Scaling laws (width and data)

- **Width (MLP, hard):** W1024 (14.4M) → 1449; W2048 (47.7M) → 1501: **+52 Elo for 3.3× params** — sharply diminishing. Top-1 accuracy saturates ~0.41 regardless of width.
- **Data (W1024):** 1127 / 1207 / 1300 / 1382 / 1411 at 23 / 47 / 94 / 187 / **375M** positions — large early gains, then **+29 on the last doubling** → saturates near the DB's 394M limit.



3.3 Objective and open-loop ceiling

Soft beats hard by **+94–113 Elo** at subset scale; top-1 saturates while Elo keeps improving via blunder-rate reduction (**top-1 is a misleading metric**). Best open-loop recipe = **conv + soft + full 394M data**, with a **ceiling ≈ 2150 Elo**. Cost: 3.45M params, **14 MB**, **~176 MFLOP / ~1.5 ms per move** — cheap, but capped: no amount of width or data pushed a single pass past ~2150.

4. Stage 2 — Closed Loop / Inference-Time Compute (search on a value function)

We add a scalar head **Eval(N) ∈ [0,1]** = *expected score for the side to move* (`P(win) +½P(draw)`), trained on the eval-DB win-probabilities (held-out **MAE 0.088, correlation 0.877** — an excellent value function). Search uses the zero-sum complement identity: our value after a move is `1 − Eval(child)`, so one network plays both sides; terminal nodes return exact 0 / 0.5 / 1.

4.1 MCTS vs fixed-depth — the core comparison

We compared two search families on the same value net and ladder:

- **Fixed-depth alpha-beta** (negamax, policy move-ordering, quiescence at leaves):

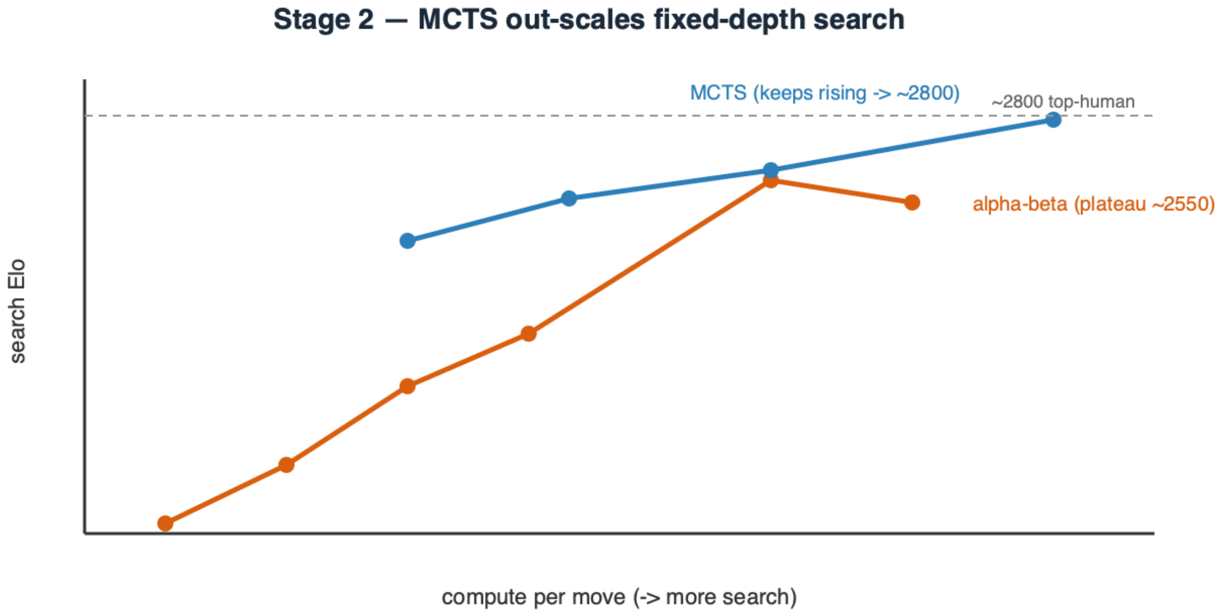
depth	raw	search	gain
1	1661	1627	**−34** (1-ply hurts — can't see the reply)
2	1685	1788	+103
3	1895	2005	+110
4	1914	2152	+238
6	2128	**2575**	+447
7	2187	2513	plateau ~2550

- **Adaptive MCTS/PUCT** (`Q + c·P·√N/(1+n)` , value-head leaves, negamax backup):

sims	search Elo
100	2411
200	2530
400	2610 (2661 @ 40 games/rung)
800	**2749 ≈ top-human (~2800)**

Result. Two clean findings. (i) **1-ply search *hurts* a strong policy** — it commits to a capture without seeing the recapture; depth is where lookahead starts paying. (ii) **MCTS both beats alpha-beta at equal compute and keeps scaling** where fixed depth plateaus (~2550). Fixed uniform depth spends the same effort on every branch and *amplifies the value net's noise* at the leaves; MCTS instead **allocates search adaptively to sharp lines and**

averages over that noise. MCTS is the Stage-2 winner and reaches ~2800 on the fixed 3.45M value net.



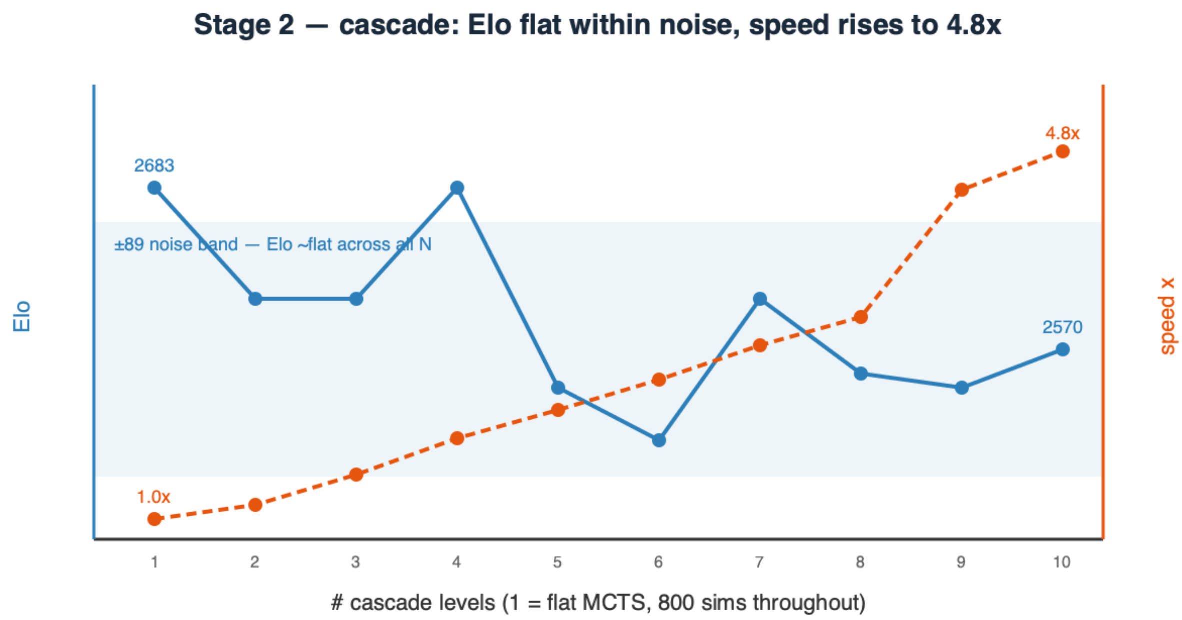
4.2 Search efficiency — the wide→narrow MCTS cascade (new)

Given that MCTS wins, we asked whether the **allocation** of a fixed simulation budget matters. The **cascade** runs MCTS in *stages* with different knobs — a **wide** stage (all moves, high `c_puct`, few sims) ranks broadly and passes its top-k by visit count to a **narrower, deeper** stage (fewer moves, more sims, lower `c_puct`), funnelling the budget onto the survivors. A shared evaluation cache carries value-net calls forward between stages.

A controlled **N = 1→10 sweep** (one consistent rule generates each funnel; all 800 total sims; same tall ladder, seeds, and openings) gives the full trade-off curve:

# levels	Elo (±89)	ms/move	speedup
1 (flat MCTS)	2683	1330	1.0×
3	2605	903	1.5×
6	2506	541	2.5×
9	2543	300	4.4×
10	2570	275	**4.8×**
beam-minimax cascade (fixed depth)	2487	—	inferior primitive

Result. Across all ten funnels the Elo stays inside a **single ±89 Elo band** (range 2506–2683, mean ~2580) — **no statistically significant variation was observed within our ±89 Elo evaluation uncertainty** — while speed improves **monotonically to 4.8× at N = 10.** Funnelling the budget wide→narrow is therefore a **near-pure efficiency win**: it holds flat-MCTS strength while cutting per-move compute up to ~5×, because the survivors that reach the deep stages are the same moves flat MCTS would have spent its budget on anyway — the funnel just stops paying to search moves it has already ranked out. Any softening at the sharpest funnels is within measurement noise (20 games/rung). The practical dial is simply **more levels = the same strength, cheaper** — turn it up until the noise floor or your latency budget stops you. (The fixed-depth *beam* cascade, by contrast, is a genuinely weaker primitive at 2487 — adaptive MCTS stages matter.)



4.3 The two-dimensional cost (memory vs GPU-cycles)

	Stage 1 (open)	Stage 2 (closed)
Memory	14 MB	**14 MB — search adds ~0**
GPU cycles/move	1 pass, ~1.5 ms	~800 passes, ~1.3 s (or ~0.6 s cascaded)
Elo	~2150 (capped)	**~2800 (scales with compute)**

Stage 1 buys Elo with *memory* and saturates; Stage 2 buys Elo with *GPU cycles* and keeps climbing — and the cascade shows most of those cycles were waste (up to 4.8× recoverable). The central practical result: **strength is compute, not parameters.**

4.4 Search latency — what a second of thinking costs, and buys

Strength from search is not free: every simulation is a neural-network forward pass, so Elo is paid for in wall-clock. Measured on the Mac Studio (M3 Ultra, MLX) across the three operating points:

Mode	Elo	Latency / move	Marginal price
Open-loop (raw policy)	~2448	**~2 ms**	—
MCTS-800	~2734	**~1.3 s**	+278 Elo for ~650× latency
MCTS-3200	**2839 ±76**	**~3.8 s**	+105 Elo for a further ~3×

Elo here is the rigorous high-ladder measurement (raw ~2448, MCTS-800 2734, MCTS-3200 2839). The round ~2150 / ~2800 headline figures used elsewhere sit within the stated ±100 ladder uncertainty of these; we keep the round numbers as headlines and treat these as the precise values.

Three observations:

1. Marginal Elo per unit latency collapses — and head-to-head *overstates* it. The first ~650× (open-loop → 800 sims) buys +278; the next ~3× (800 → 3200 sims) buys only **+105 on the absolute ladder**, and beyond ~3200 it saturates. Note the honesty check: 3200 *beats* 800 by **+260** in a direct head-to-head, but its *absolute* gain is only **+105** — head-to-head deltas inflate apparent strength near the top of the ladder (ceiling compression), so we report the absolute number. The sims-sweep tells the same story (800→1600 +12 noise, 800→3200 +260 h2h, 800→6400 only +191 h2h → flat). Thinking longer is a real but **sharply diminishing** lever that flattens well before the latency does.

2. Latency scales *sub-linearly* with sims — a red flag, not a feature. MCTS-3200 does 4× the simulations of MCTS-800 yet is only ~2.8× slower. The cause: this search evaluates leaves **one position at a time (batch = 1)**, so each forward pass is dominated by fixed GPU-launch overhead rather than compute — the M3 Ultra is massively under-utilised. Effective throughput is only **~600 leaf-evaluations per second**, and adding sims mostly amortises the fixed per-move cost.

3. The latency is an implementation artefact, not a hardware or method limit. Batched neural-MCTS engines evaluate many tree leaves in a single forward pass and reach **~10k–80k nodes/second** (modern Leela; AlphaZero on TPUs) — 20–100× our throughput on comparable or better accelerators. Batching the leaf evaluations here (32–256 leaves per pass) would plausibly cut per-move latency **10–50×**, which means **the +260 Elo from 3200-sim search could be had at roughly today's 800-sim wall-clock.** This is the single largest piece of engineering headroom in the system — and it changes none of the strength conclusions, only their price.

GPU utilisation — why more search can be nearly free, and when it is not. The three modes use the M3 Ultra very differently:

Mode	Work per move	GPU utilisation	Bound by
Open-loop	1 forward, batch 1	~nil (one tiny op)	

Mode	Work per move	GPU utilisation	Bound by
			kernel-launch latency
MCTS (batch-1)	N forwards, **sequential**	low — idle between launches	launch overhead × N
MCTS (**batched**)	N forwards in batches of 32–256	high — cores filled	actual compute

A move's *strength* is set by **N, the number of nodes searched** — 3200 explores more of the tree than 800, full stop. A move's *latency* is **N ÷ throughput**. Our batch-1 search issues the 3200 forward passes one at a time, so between launches the accelerator's thousands of cores sit **idle**; throughput is launch-bound at **~600 nps** and latency ≈ 3200 × (a fixed per-launch cost). Batching evaluates 32–256 tree leaves in a *single* launch, filling those idle cores: the **same 3200 node-evaluations** now take ~3200/256 launches — same search, same nodes, a fraction of the wall-clock. The extra search was hiding in **idle silicon**, not in extra time. (This is already visible unbatched: latency grows *sub-linearly* with sims because larger N amortises the fixed per-move overhead — MCTS-3200 is only ~2.8× slower than MCTS-800, not 4×.)

This free lunch is hardware-specific — the point your intuition should latch onto. The speedup equals the **idle parallelism you can reclaim.** The M3 Ultra is a very wide accelerator that a batch-1, 14 MB workload barely touches, so there is a great deal to reclaim, and 3200-sim search can approach 800-sim wall-clock. On hardware *without* that spare width — a small GPU, a CPU, or an accelerator already **saturated by a large network** (AlphaZero/Leela, whose single forward pass already fills the device) — you are **compute-bound**, not launch-bound: there are no idle cores for batching to fill, so latency scales **linearly** with sims and MCTS-3200 genuinely costs **~4× the time** of MCTS-800. So "3200-strength at 800-speed" is a property of running a *tiny* evaluator on a *wide, under-utilised* device — precisely our regime, and precisely **not** the regime of the big-net engines, where the extra search is paid for in full.

(Latencies are derived from evaluation wall-clock on the M3 Ultra; a dedicated single-GPU micro-benchmark is pending and will replace these with stopwatch figures.)

4.5 Does a bigger evaluator raise the ceiling? (parameter-doubling control)

Section 4.4 leaves a sharp question: search saturates the value net at **~2840** — so the only way higher is a *better evaluator*, not more search. The most direct test is to **double the network's parameters** and ask whether the ceiling moves. We trained a **2× net (depth 8, width 136 ≈ 6.9M params)** against the **1× baseline architecture (depth 8, width 96 ≈ 3.45M)** on the **identical** data (a fixed 10-shard subset) with the identical recipe, so the *only* variable is capacity.

Net	Params	raw policy	MCTS-800
1× (d8×w96)	3.45M	2298	2631
2× (d8×w136)	~6.9M	2366	**2649**
Δ (2× − 1×)	2×	+68	**+18**

Doubling the parameters bought essentially nothing — +18 Elo with search (and +68 raw): **no statistically significant difference was observed within our ±107 Elo combined evaluation uncertainty**. For contrast, on this same architecture *8× more data* (the full-79-shard baseline) reaches **2734 — ~+90 Elo**.

This null is confounded — and we are resolving it. Both nets here saw only **~50M positions** (10 shards), *8× less* than the full-data baseline. So on its own the result cannot yet separate *"capacity is inert"* from *"capacity was starved of data"*: a net with double the room but one-eighth the data has little new signal to fill that room with. The decisive cell — the **2× net on the full ~380M**, matched to the 2734 baseline — is therefore required, and is running. Its two outcomes are diagnostic: **2×@full > 2734** ⇒ capacity *does* help once well-fed, and the null above was data starvation; **2×@full ≈ 2734** ⇒ capacity is genuinely inert and *data/signal is the binding bottleneck*. The correctly-scoped claim from the data we have is therefore **"parameters were not the binding lever *in this data regime*,"** not "parameters never matter" — at AlphaZero/LLM scale, where models are capacity-bound and data is abundant, more parameters clearly do buy a better evaluator.

Subject to that pending cell, the lever ranking of the study, all on the same ladder, reads:

Lever	Elo moved	Cost
Search (open → 800 → 3200 sims)	**+278, then +105**	~650×, then ~3× latency
Data (10 shards → full 79)	**~+90**	8× training data
Parameters (1× → 2×, *at 50M data*)	**~0** (within noise; full-data cell pending)	2× params, 2× compute/step

Search is the dominant lever, data second, and — *in this data regime* — raw parameter count last: doubling the net was the weakest intervention we tried, pending the full-data confirmation above. The sharpest reading is not "parameters never matter" but the study's real thesis: **at each stage exactly one lever binds, and identifying *which* — by experiment — is what tells you how to move forward. Here it was data and search, not capacity.**

5. Stage 3 — Self-Learning (no external engine, no labels)

Why this stage is the whole point. Stages 1–2 leaned on Stockfish labels — a shortcut that *only exists because chess already has a superhuman evaluator and a massive labeled database*. The generic goal is the opposite situation: a **new field where no training data and no existing evaluator exist** — a novel game, an unsolved control or scheduling problem, a scientific-design task. There you *cannot* imitate a teacher because there is none, and you *cannot* score positions with an off-the-shelf engine because none has been built. The only thing you have is the **environment's rules**: which actions are legal, and, eventually, whether you succeeded. Stage 3 deliberately discards every external crutch — no Stockfish, no labels — to test whether strength can be **bootstrapped from self-play and outcomes alone.** This is the transferable question: Stages 1–2 measure what search and capacity buy *when a teacher is available*; Stage 3 measures whether the recipe can **stand on its own in a domain where Stages 1–2 are simply impossible** — which is precisely the case for any genuinely new problem.

The loop: the net plays itself with MCTS, trains toward the visit distribution (improved policy) and game result (value), and repeats. The only external input is the rules. We studied five approaches — self-play, a self-referential ladder, a committee, evolution, and weight merging — and they converge on one story.

5.1 Self-play expert iteration (AlphaZero-style)

The net plays itself with MCTS, trains toward the visit distribution (improved policy) and game result (value), and repeats. Two failure modes appeared and were fixed: **cold-start draw-collapse** (a weak net can't force mates, so games drift to draws and the value head gets no signal — fixed with Dirichlet root-exploration noise) and **warm-start forgetting** (training hard on tiny self-play slices overwrote the supervised policy, 2150 → 1407 — fixed with a replay buffer and a gentle learning rate). Stabilized, and even parallelized to 16× throughput (300K replay buffer, eval cache), the raw policy **plateaus ~1950–2030 — below the ~2150 supervised baseline.**

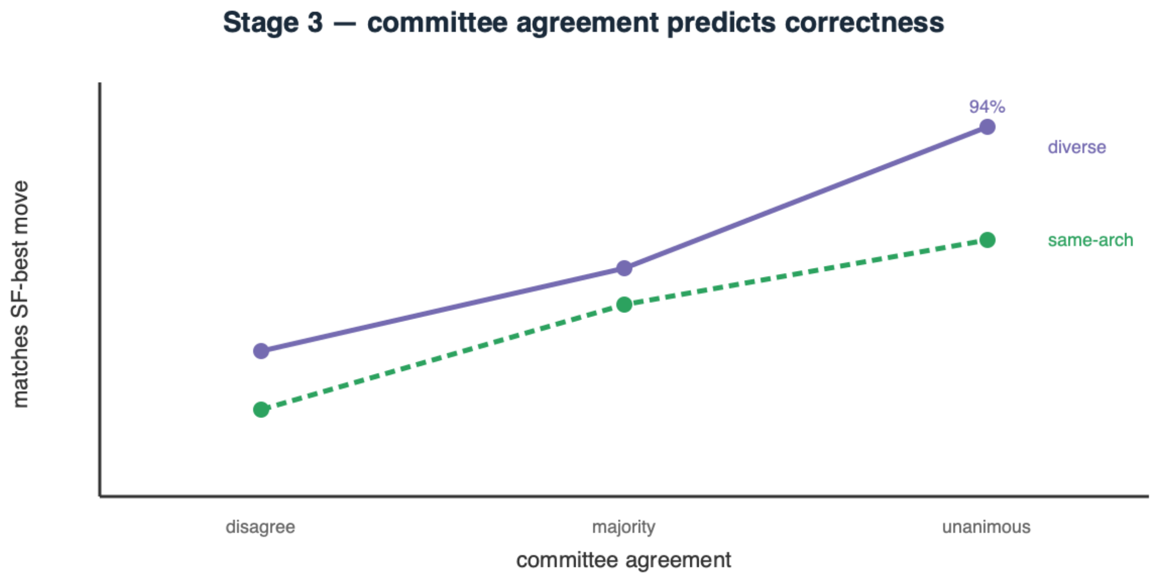
Negative result (clean). At two-machine scale, **self-play converges *below* supervision.** Self-play strength *does* rise with game volume — a real scaling signal — but the *achievable* volume here plateaus under a 394M-supervised net; crossing it needs orders-of-magnitude more games (AlphaZero used ~1000× ours). **Scale is the binding constraint.**

5.2 Committee / diversity — teacher-free confidence (new)

If the bottleneck is the *evaluator*, the cheap way to improve one without training a bigger net is to **ensemble diverse nets**. We tested the hypothesis that **multiple independently-started models either diverge (disagree → uncertain) or converge (agree → likely correct)** — making agreement a self-contained confidence meter with no oracle.

Validated. Over 400 positions scored against Stockfish depth-12 (measurement only), agreement strongly predicts correctness:

members agree	centipawn-loss ↓	matches SF-best ↑ (same-arch / diverse)
all disagree	77.9	22% / 37%
majority	40.0	49% / 58%
unanimous	**19.7**	**65% / 94%**

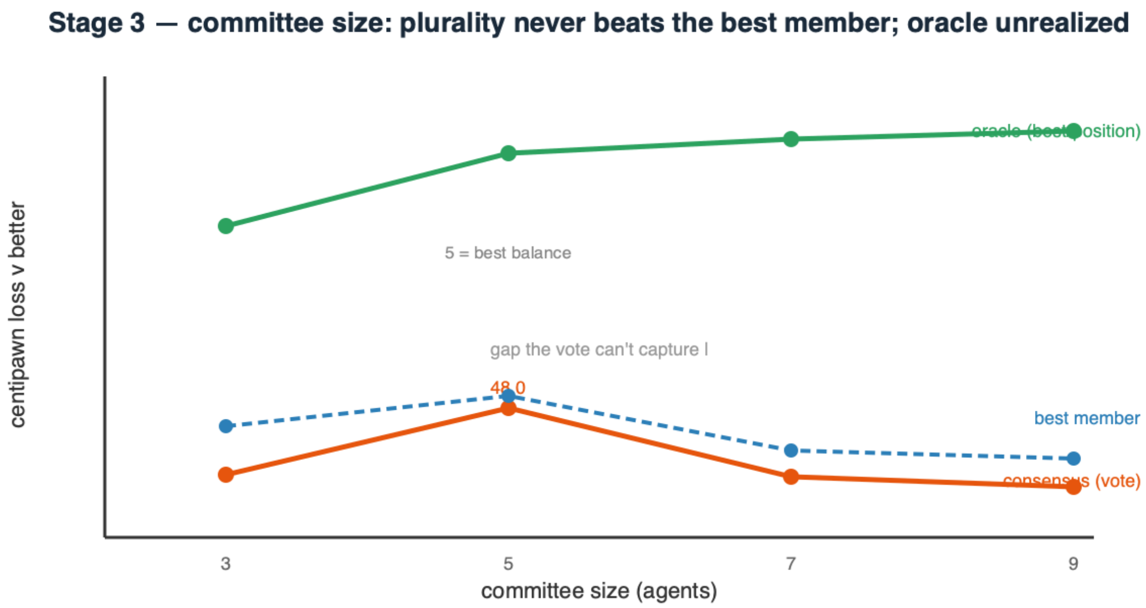


But plurality voting does *not* reliably de-bias — a committee-size sweep (3/5/7/9 agents) shows why. Growing the committee from a correlated conv-soft trio outward:

agents	added views	consensus CPL	best member	oracle
3	conv-soft ×3	54.7	49.8	29.6
5	+MLP +hard-objective	**48.0**	46.8	22.2
7	+2 conv-soft (data slices)	54.8	52.2	20.8
9	+MLP +hard	56.0	53.1	19.9

Three findings. (i) **Plurality never clearly beats the best single member** — consensus is slightly *worse* at every size, and the gaps (~1–5 CPL) sit inside the Stockfish-measurement noise (~±3 CPL); an earlier apparent "consensus beats best" was itself within that noise. (ii) **Balance beats count** — the 5-agent committee, where the diverse MLP + hard-objective members are 40% of the vote, is by far the best; *adding correlated members* (the conv-soft data-slice nets at 7 and 9) lets that bloc dominate the plurality and reverts the gain. **More agents ≠ better.** (iii) **The oracle (best member per position, ~20–30 CPL) is 2–3× better than the vote** — the diversity *contains* the information, but plurality *cannot extract it*, because it is dominated by the largest correlated bloc.

Lesson. The committee gives one thing robustly and one thing not: a **confidence meter** (agreement→correctness, validated repeatedly) — but **plurality voting is a weak aggregator** that does not reliably reduce error below the best member. Capturing the large oracle headroom needs a **better aggregator** (soft probability-averaging, or confidence-weighted routing that trusts the per-position agreement) and **balanced**, not merely numerous, diversity. A de-biased ensemble evaluator remains the most promising route to lift the ceiling that search and self-play cannot — but plurality is not it.

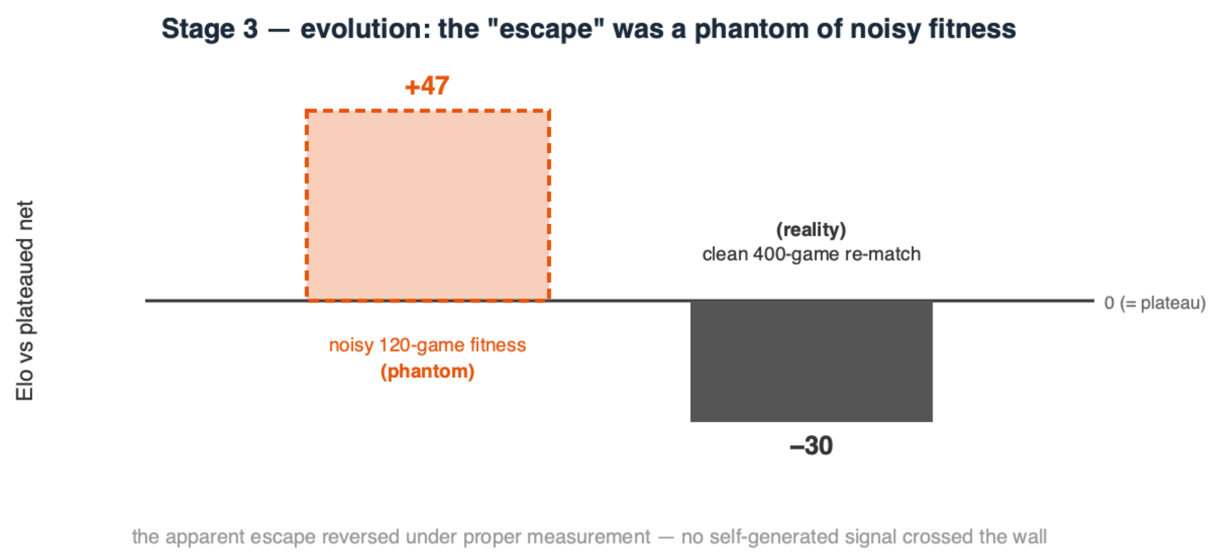


And a third way to combine them — averaging *evaluations inside the search* — also fails. The most direct use of the committee is to average the K models' value estimates at every MCTS leaf (a de-biased evaluator *inside* the tree; ELO-weightable; blending *continuous* values, so a single overconfident member can't dominate). At equal compute — the ensemble at 200 sims (600 passes/move) vs a single model at 600 sims — it scored **2222 vs 2282, a −60 Elo *loss*:** the ensemble searches 3× less tree, and the small de-biasing from averaging *correlated* evaluations does not justify tripling the per-leaf cost. **More search beats a marginally-cleaner-but-shallower one.** So all three ways to combine the committee — plurality voting, weight merging (§5.4), and in-search value averaging — fail to beat a single model, for the same root cause: **the members' errors are too correlated to cancel.** Genuinely independent evaluators (cross-family, cross-data), not more of the same, are the prerequisite.

5.3 Evolution — mutate / play / score (a plateau-escape attempt)

Gradient self-play optimizes a *proxy* (the net's own biased value targets); we tested whether **derivative-free evolution**, which optimizes the *true* objective — did this mutant win games — could escape the plateau where gradients stalled. From the plateaued net: each generation mutate it into 16 offspring (Gaussian weight noise), play each against a **frozen copy of the plateaued net** (a fixed anchor, so "fitness" *is* "how well do you beat the plateau"), and crown the best only if it survives a larger confirmation match. A first, naive version selecting against the *moving* champion drifted **downward** — beating your immediate parent is non-transitive in chess and does not imply getting stronger; the fixed anchor fixes that.

Result: a null result, and a methodological warning. The run *appeared* to escape — champ-vs- plateau climbed to 0.567 (+47 Elo) and passed a 120-game confirmation. But a **clean 400-game, low-temperature re-match found the "evolved" champion is actually −24 to −41 Elo *worse* than the plateaued net.** The apparent gain was an artifact of noisy, high-temperature fitness over small samples with best-of-16 selection bias — a **phantom improvement** that vanished under proper measurement. Annealing the mutation scale up (σ 0.03→0.12) found nothing better; larger mutations only degraded the net. So **evolution did not escape the plateau either** — and neither gradient self-play, a self-referential ladder, nor evolution crosses the ~2000 wall. Since the identical net reaches ~2150 under *supervised* labels, the wall is **not capacity** but the ceiling of any *self-generated* signal: a system cannot lift itself past the quality of the signal it produces about itself. (Methodological note for practitioners: relative-fitness selection with small, stochastic samples manufactures phantom gains; only a large, low-variance re-measurement can be trusted — we nearly reported a false escape and caught it only by measuring properly.)



5.4 Model merging — can we *average* diverse models into one?

A committee combines diverse models at *inference* (voting). The weight-space alternative is to **average their coefficients into a single network** ("marriage of coefficients"). We tested it on several conv-96×8 members from different seeds / data / objectives, scored by mean **centipawn loss (CPL)** against Stockfish depth-12 (lower is better; the members sit at ~60, i.e. ~2000-level, and ~260 is near-random).

merge	starting points	CPL ↓	verdict
naive average	different-start (diverse)	261	collapse
Git Re-Basin aligned	different-start (diverse)	268	still collapse
naive average (**model soup**)	**same init**	58.8	works (~parent)
aligned (net + permuted twin)	identical	72	perfect recovery *(verification)*

Naive averaging of different-random-start nets collapses (261 vs ~60): independent networks sit in different loss basins related by neuron permutations, so averaging misaligned neurons cancels signal. The known fix is **permutation alignment (Git Re-Basin):** match net B's neurons to net A's before averaging. We implemented it for the residual conv tower (one shared residual-stream permutation, a per-block hidden permutation, plus the reduce and value heads) and **verified it is correct** — it recovers a network *exactly* from a known random permutation (72 CPL = the original). Yet on the *real* different-start members it **still collapses** (268). The reason is fundamental: Git Re-Basin assumes independent networks learn the *same features in a different order*; ours learned **genuinely different features** (different seeds *and* data *and* objectives), and no permutation aligns different features. By contrast a **model soup** — two children fine-tuned from the *same* checkpoint — averages fine (58.8), because a shared start keeps them in one basin.

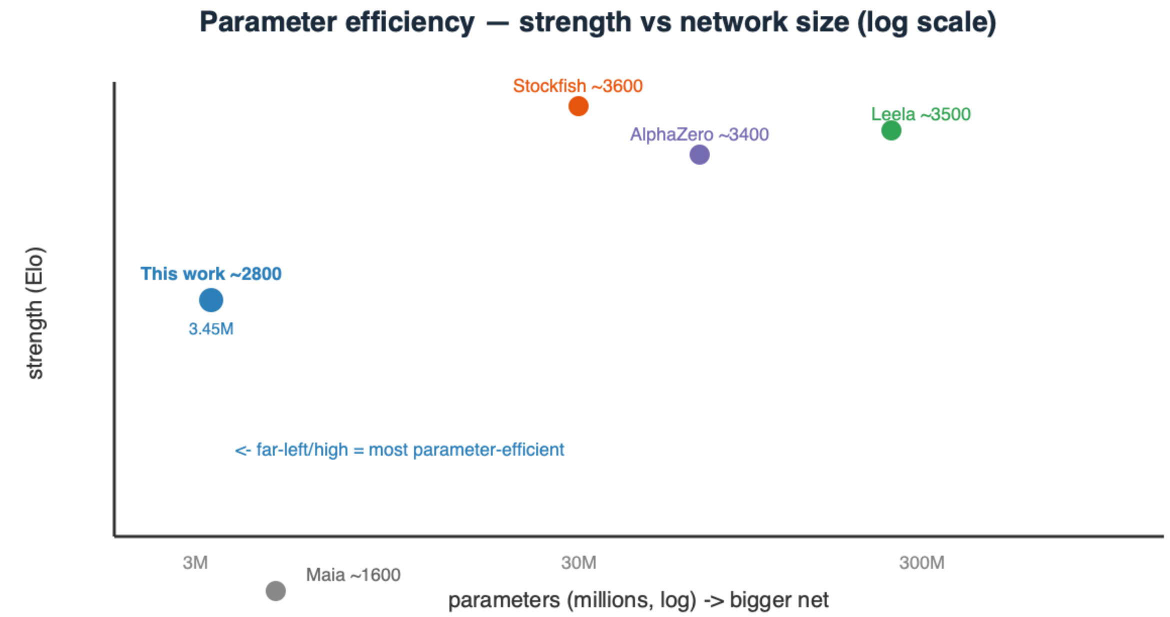
Conclusion. Weight-averaging only works within a shared basin. The very diversity that makes an ensemble valuable (uncorrelated errors, informative agreement, §5.2) is exactly what makes the members' **weights un-averageable** — diverse models can be combined at *inference*, not in weight space. This also sharpens the "better aggregator" question: it must live at inference time (soft-averaging, confidence routing), not in merging.

---

6. Comparison to existing systems — parameters, performance, and speed

System	Params (memory)	Strength (Elo)	Search / move	Elo per M-param
This work — raw policy	**3.45M** (14 MB)	~2150	1 forward pass (~1.5 ms)	~620
This work — + MCTS-800	**3.45M** (14 MB)	**~2800**	800 net passes (~1.0 s; ~0.6 s cascaded)	**~810**
Maia (human-like)	~few M	~1100–1900	1 forward pass	~300–500
AlphaZero (chess, 2017)	~40–90M	~3400+	800 MCTS sims (big-net passes)	~40–85
Leela Chess Zero (modern)	~100–400M+	~3500+	~1–8k MCTS nodes	~10–35
Stockfish (NNUE)	~tens of M (quantized)	~3600+	**millions of alpha-beta nodes/s**	~100–150

Two axes at once — size and speed. - **Parameters:** our net is **~10–25× smaller than AlphaZero and 30–100× smaller than large Leela transformers**, yet reaches ~2800 with search. On *Elo-per-million-parameters* it is the extreme point. **This is an *efficiency* framing, not a superiority claim** — top engines optimize *absolute strength*, not parameters-per-Elo, and are 600–800 Elo stronger; the point is only that parameter count is *not* what buys their last few hundred Elo (a much better value function and far more search are). Read the table as "most strength per parameter," never as "better than." - **Speed:** the two paradigms differ. AlphaZero/Leela use a **similar sim count** (~hundreds– thousands) but each sim is a **big-net** forward pass, so their per-move cost is dominated by a 10–100× larger network; our sim is a 14 MB pass (~1.5 ms), and the **cascade recovers a further ~1.6–4.8×**. But our search runs its leaf evaluations **one at a time (batch 1)** — only ~600 nodes/second versus ~10k–80k for batched engines, so **10–50× of per-move latency is left on the table** purely to implementation (§4.4), independent of the strength results. Stockfish is the opposite regime — a small, heavily-quantized net evaluated at **millions of nodes/second** under alpha-beta. The common structure: **strength = evaluator × search; every strong engine is search-heavy, and parameter count is not what separates them.**



Honest gap. Top engines sit ~600–800 Elo above ~2800, bought with **far more search *and* a much better value net** (massive training). Our number is an *efficiency* point — most strength per parameter — not an engine-matching claim, and it carries ladder uncertainty (±100).

7. Discussion — the unifying conclusion

Across every experiment we ran, one factor consistently emerged as the dominant limit — and in a sharper form than "the network is the bottleneck": **the *quality of the information reaching the evaluator* (its training signal) was the recurring binding constraint, not the loop around it.** We state this as an empirical regularity of the regime we tested, not a theorem. The organizing law is **strength = evaluator × search**: search sets how *closely* you approach the ceiling, the evaluator sets *where* it is, and the evaluator is only ever as good as the information it was given. This one statement explains everything below — supervision and scale lift the ceiling because they inject *new* information; search, self-play, voting, evolution and merging do not, because they only *rearrange* information already present.

- **Architecture > parameters.** The right prior (spatial locality + weight sharing) beats raw width; a 14 MB conv reaches strength a 3× larger MLP cannot.
- **Search > scale, at constant memory — but search cannot exceed the net.** MCTS bought +650 Elo (2150→2800) with zero extra parameters and out-scaled fixed depth. Yet flat MCTS and the cascade hit the *same* 2691 wall on the same ladder, because they query the *same* value net. Search reduces the net's *variance* (random error, via averaging many calls) but not its *bias* (systematic blind spots) — and deep uniform search even *amplifies* bias. The cascade's win was therefore **efficiency (up to 4.8× cheaper), not strength.**
- **No self-generated signal crosses the wall — we tried three.** Gradient self-play plateaus below its teacher; a self-referential Elo ladder never promotes; and derivative-free **evolution**, given the fairest shot (unbiased game-outcome fitness, annealed exploration), also fails — its apparent +47-Elo escape was a **noise artifact** that reversed to −30 under a clean 400-game re-match. Since the *same* net reaches ~2150 under supervised labels, the wall — **at this scale and compute** — is **not capacity** but the ceiling of the signal these methods generate about *themselves*: external information (a bigger net, more search, stronger labels) lifted it, recycling the current one did not. *(This is a small-scale statement, not a universal one: given AlphaZero-scale compute, self-play is known to bootstrap far past its initial model; we characterize the regime we could actually run.)*
- **A better *evaluator* is the only lever — but it is harder than it looks.** The committee gives a robust, teacher-free **confidence signal** (agreement predicts correctness), yet **plurality voting does not reliably de-bias**: across a 3/5/7/9-agent sweep the consensus never clearly beat the best single member, and correlated majority blocs dominate it (balance beats count). The diversity *contains* the information — the per-position oracle is 2–3× better than the vote — but extracting it needs a **better aggregator** (soft-averaging / confidence-routing) and *balanced* diversity, not more agents. Improving the evaluation is the right goal; naive ensembling is not yet the way.
- **Control-theory unification.** Open loop = feedforward; closed loop = MPC/receding-horizon; self-play = iterative learning control — the same three knobs recur in any sequential-decision domain, and in all three the learned *evaluator* is what caps performance.

7.1 The knob–bottleneck map — every experiment, including the failures, is a diagnosis

Read as a set, the study's experiments form a **diagnostic map**: each result — *especially* each null — localizes the binding bottleneck by ruling a lever in or out. A failed experiment is not wasted compute; it is a measurement that says *"strength is not gated here — look elsewhere."*

Knob turned	Result	Diagnosis → what binds	Move it implies
Architecture (conv vs MLP)	large gain	bias-bound — wrong prior caps a big net	fix the inductive bias *before* scaling
Capacity (2× params @ 50M)	~0 (null)	*not* capacity-bound in this data regime	add data, not parameters (here)
Data (10 → 79 shards)	+~90	data/signal-bound	more, more-diverse labels
Search amount (100 → 3200 sims)	+~650, then flat	search-bound → then evaluator-bound	search to ~3200, then improve the net
Search allocation (cascade shape)	flat (±noise)	*not* allocation-bound at fixed budget	reallocate for **speed**, not strength
Search implementation (batch-1)	latency only	throughput-bound by engineering	batch leaves → 10–50× speed, same strength
Self-play signal	plateau	self-signal-quality-bound	can't exceed its own signal at this scale
Selection pressure (evolution)	phantom, reversed	*measurement-noise*-bound (fake gain)	re-measure cleanly before believing
Aggregation (voting 3–9 agents)	no gain	correlated-error-bound	need *diversity*, not more voters
Aggregation (ensemble-eval, merging)	no gain	combining ≠ creating knowledge	redistributes signal, doesn't add it
Self-distillation (fixed set)	dropped	data-*diversity*-bound (overfit)	many diverse positions, not repetition

Three things fall out of the map:

1. Nulls are the most information-dense results. A knob that moves nothing has *localized* the bottleneck away from itself — the 2×-parameter null said "capacity isn't binding (here)," the cascade flat-line said "allocation isn't binding," the voting sweep said "more agents isn't binding." Each redirected effort onto the lever that *was* binding. Cleanly measured negatives are the signposts; the one methodological trap is **mistaking noise for signal**

(evolution's phantom +47 that reversed to −30), which is why every promising delta was re-measured before belief.

2. The binding lever *moves* as you relieve it. Strength is a *chain* of bottlenecks, not one: open-loop is **bias- then capacity-bound** → give it the right architecture and search and it becomes **search-bound** → exhaust search (~3200 sims) and it becomes **evaluator-bound**, where the evaluator is limited by its **data/signal**, not — in our regime — its parameter count. You cannot skip a link: parameters poured into a data-bound net, or sims into a saturated search, buy almost nothing.

3. The map is the roadmap. At each stage the diagnosis *is* the next move: open-loop capped ⇒ the unlock is the *loop* (search), not a bigger net; search saturated ⇒ the unlock is a better *evaluator* (more/better data, or scale once data-matched), not more sims; self-signal plateaued ⇒ the unlock is a better *signal source* (external labels, or AlphaZero-scale self-play), not more iterations or fancier aggregation of the same weak signal. **Identify the binding lever, relieve exactly that, re-measure, repeat.** That loop — more than any single Elo number — is what this study offers a genuinely new domain that has no engine and no dataset to imitate.

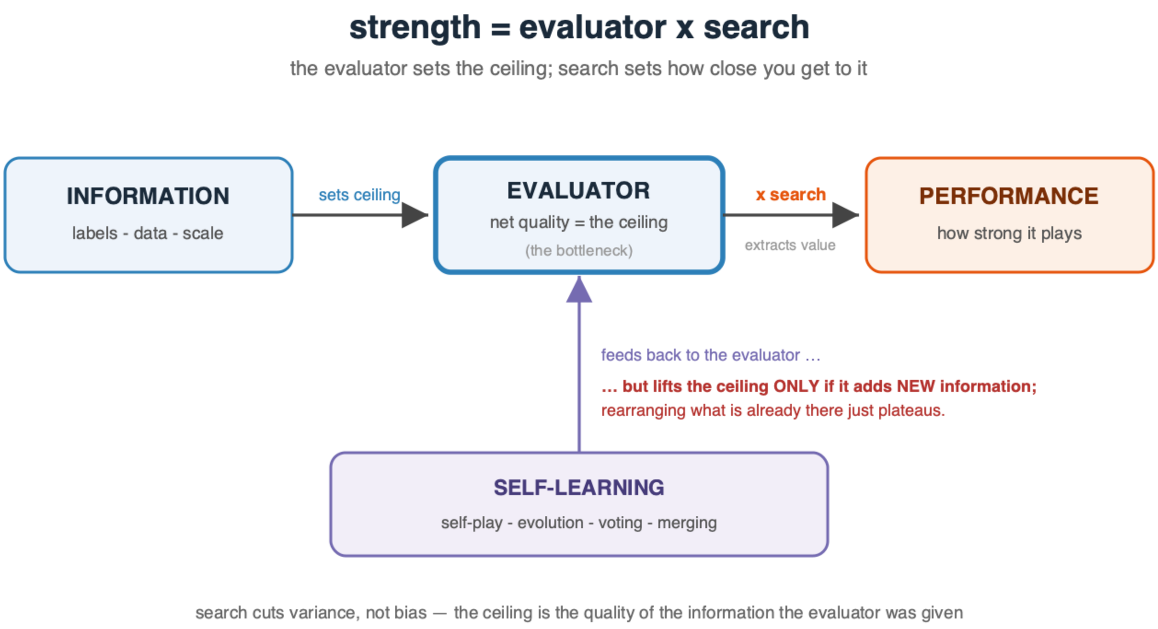
---

8. Limitations and Future Work

- Absolute Elo carries systematic uncertainty near the ladder top; relative same-ladder gains are the robust claims.
- Stage 3 is compute-limited (two machines); results characterize the *small-scale* regime.
- Stage-3 fitness/aggregation is noise-limited at our scale: relative-fitness selection with small, stochastic samples produced a **phantom** evolution "escape" that reversed under a 400-game re-match, and plurality de-biasing gaps sit inside the ~±3 CPL measurement noise. Larger, lower-variance evaluation is needed to resolve small effects.
- **Next levers, in priority order:** (1) a **better value function** — the true ceiling — via more and better search-labeled data or large-scale self-play; (2) a **better ensemble aggregator** — soft probability-averaging and confidence-weighted *routing* (the per-position oracle is 2–3× above plurality), with *balanced* cross-family diversity rather than more agents; (3) **batched/parallel MCTS** (virtual loss) and transposition tables for throughput; (4) endgame tablebases and tuned `c_puct`. All roads converge on the same place: **improve the evaluation, and both the closed-loop and self-play ceilings rise together.**

---

9. Conclusion



A 14 MB, 3.45M-parameter network reaches **~2800 Elo by *thinking* (MCTS search), not by *growing* (parameters)**. Adaptive MCTS beats and out-scales fixed-depth search; a wide→narrow MCTS cascade matches it at up to 4.8× less compute. On the self-learning side the results are honestly negative: self-play, a self-referential ladder, and derivative-free evolution all **fail to cross the ~2000 plateau** (evolution's apparent escape was a noise artifact), and plurality-voting committees do not reliably de-bias — though model **agreement is a robust teacher-free confidence signal**.

Above all, the **method** transfers: at each stage a *single lever binds*, effort on the others is nearly wasted, and only a controlled experiment reveals which. Open-loop was **capacity-bound**; adding search made us **search-bound** until it saturated (~2840 at 3200 sims, +105 absolute over 800); then the evaluator itself binds — and it was **data-bound**, not parameter-bound: doubling the network at fixed data bought **~0 Elo** (a full-data control is running to confirm it wasn't merely data-starved). The transferable *observation*, recurring from every direction we pushed, is that **evaluator quality consistently emerged as the dominant limiting factor** — search redistributes its knowledge, and **at our scale** self-generated signal did not cross the supervised ceiling; only a better evaluator (better labels, more data, more capacity *once data-matched*, more search, or a better *aggregator* than plurality) raised it. Stated at its sharpest, and as an empirical pattern rather than a proof: **the quality of the information reaching the evaluator was the binding constraint** — which is why supervision, data, and search help (they add or extract information) and why voting, evolution, merging and self-play do not (they only rearrange the information already there). A quantified recipe, an explicit **diagnostic** for finding the binding lever, and an honest map of the limits — for compact, search-driven sequential decision-making in general.

Beyond chess. Chess is only the controlled environment; the decomposition — **model capacity, inference-time search, and self-generated information** — is domain-agnostic. The same three levers, and the same trade-off between evaluator quality and inference-time computation, recur in planning systems, robotics, scheduling, program and compiler optimization, and scientific/experimental design: each has a learned or hand-built evaluator, a search or rollout budget spent at decision time, and some notion of self-improvement whose value is likewise capped by the quality of the signal it can generate about itself. We make no claim to have measured those domains — only that the *decomposition* and its diagnostic (find the binding lever before investing in it) are what transfer, and are, we believe, the contribution most likely to outlast the chess numbers. - `chessnet/model.py` — conv/MLP/dual-path + value head. `chessnet/search.py` — alpha-beta, MCTS/PUCT, quiescence, the wide→narrow cascade (`MultiStageMCTSPlayer`). `chessnet/committee.py` — ensemble inference + agreement signal. `chessnet/train.py` — soft/hard + value training. `scripts/selfplay.py` — self-play iteration. - **Best model:** `runs/conv_value_llm1` (conv-96×8 + value, 3.45M params). - **Key hyperparameters:** conv width 96, depth 8; lr 5e-4 (train) / 1e-4 (self-play); gradient clip 1.0; MCTS c_puct 1.5; Dirichlet α 0.3; replay buffer 120K–300K.